

Complementary Threat Knowledge in Federated Android Malware Detection

Attributing Gains to Peer Threat Exposure

An Android malware detector can miss a family because its own training data contain no examples of that family. Federation appears to offer a natural remedy: other participants may have seen the malware. Yet an improvement after federated training does not by itself show that the peers supplied knowledge of the missing family. A shared model also sees more benign applications and more examples of other malware, either of which may improve detection without any peer holding that particular family. This distinction matters when deciding whether a security team needs broader collaboration, targeted sharing of scarce threat evidence, or a better representation of the applications it already observes.

The chapter asks a narrower question than whether federated learning improves average Android malware accuracy: *when a family is absent from one client's training data, what part of that client's gain is attributable to peers' examples of that family?* “Unseen” here means unseen by the evaluated client during training, not a globally new or zero-day family. The detector remains binary (benign versus malware); family labels define the evaluation strata and exposure intervention. The chapter will make this unit of analysis explicit so that a family-specific benefit is not confused with a new family-classification method.

Prior studies establish important pieces of the problem. [Ongun et al. \(2022\)](#) report a local-to-federated advantage through CELEST for malware observed at a peer network, albeit in HTTP traffic rather than Android applications. [Shen et al. \(2024\)](#) evaluate attack classes removed from individual intrusion-detection clients in FLEKD-IDS. [Feng et al. \(2025\)](#) vary how many IoT clients lack a malware in CyberForce, including a condition in which all lack it, and examine mitigation performance under several aggregators. These studies make it untenable to claim that transfer to a client missing a threat, or an absent-everywhere comparison, is itself new. What remains difficult to identify from their reported contrasts is the family-specific increment over the benefit of pooling otherwise comparable data. In the accessible work reviewed for this proposal, a volume-matched absent-everywhere control paired with a peer-present condition and a local baseline has not been reported for Android malware detection, particularly with evaluation on the deprived client's own domain. The contribution is this attribution design and the evidence it yields, rather than a new federated optimizer.

Chapter focus and comparative design

CTK-Android treats heterogeneous application-market and era partitions as simulated federated clients. For each eligible client–family pair, the target client's training partition contains no examples of that family while one or more peers can retain them. Three comparisons organize the chapter: a local-only detector; a collaborative detector trained with the family absent from all clients; and a collaborative detector trained with the target family present at peers. The two collaborative arms are matched in training volume. Their difference estimates the incremental value associated with peer-held family evidence under the tested representation, learner and sampling design. The difference between local-only and family-absent collaborative training describes generic pooling benefit. A full-exposure reference helps distinguish poor recognition caused by missing examples from a limit of the tested feature and model combination. It is an empirical reference, not an optimality bound.

The application evidence comes from LAMDA static features and family labels, supplemented with AndroZoo metadata for client construction ([Allix et al., 2016](#); [Haque et al., 2026](#)).

LAMDA was assembled to study longitudinal Android malware detection; this chapter uses its family distribution to study collaborative exposure, not to claim a deployed cross-organization federation. AVClass2-derived family names are useful experimental strata but are imperfect labels (Sebastián & Caballero, 2020). Applications sharing a package or an identical feature vector are kept within one fit, calibration or test partition to reduce identity leakage. The main outcome is recall on the locally absent family at client-calibrated benign false-positive targets, with realized false-positive rates reported alongside it. Separating federation-wide test malware from the target client’s own market/era test data checks whether a gain is meaningful in the deprived client’s domain rather than driven by a different peer population. Family-level and worst-client views prevent a mean improvement from hiding failure for the clients most in need of help.

The chapter will compare local training, centralized pooling, FedAvg and other simple collaborative variants under the same exposure logic. FedAvg provides a recognizable baseline, not a claimed innovation (McMahan et al., 2017). It will examine exact peer-example doses, naturally scarce families, and a wrong-family placebo that keeps additional peer malware support while changing its family identity. The placebo helps test whether a positive contrast is specific to the family of interest. A label-permutation control tests whether the gain survives disruption of the family signal. Robustness checks cover model choices, family selections, training support and operating points. Paired uncertainty estimates and family/client support counts will accompany the comparisons; no single aggregate score will stand in for every family.

What the chapter will contribute

The chapter’s principal methodological contribution is a controlled-exposure design that separates generic federation benefit from the increment attributable to peer-held examples of a locally absent malware family. The central output is an interpretable decomposition of collaboration benefit. A positive local-to-federated difference can coexist with little family-specific transfer, while another family may benefit materially only when peers actually hold its examples. The chapter will show how to distinguish those cases, how much peer exposure is associated with a detectable increment, and whether gains persist for the target client’s own data and for poorly served clients. It will also examine families that remain difficult even under full exposure, since more collaboration cannot be assumed to repair a weak static representation or a model mismatch. Training-only feature-space similarity to known malware offers a bounded explanation for why a family label can be locally absent while its feature patterns are familiar.

This is an empirical measurement chapter with a defined threat and inference boundary. The clients are constructed domains within one Android corpus, not real independent organizations. The study does not measure differential privacy, secure aggregation, resistance to poisoning, or a privacy guarantee from federation alone. It does not test globally novel malware or dynamic behavior. The causal reading of the matched contrast is confined to the controlled exposure intervention; market and era effects, noisy family labels, and source-specific feature processing remain limitations. These constraints are part of the chapter’s value: they specify what an apparent collaboration gain can and cannot justify.

For *Advances in Mobile Application Privacy and Security*, the chapter joins two topics that are often discussed separately: mobile malware benchmarking and federated security analysis. Readers will receive a concrete evaluation design for deciding when peer threat evidence matters, together with family-aware, operating-point-aware evidence about its benefits and failures. The chapter will proceed from the attribution problem and closest prior contrasts to the exposure protocol, results by family and client, robustness checks, and implications for mobile threat-sharing practice. Its practical conclusion will be conditional: collaboration is valuable when it supplies evidence a client lacks and when the tested detector can use that evidence; total federated gain alone cannot establish either condition.

References

- Allix, K., Bissyandé, T. F., Klein, J., & Le Traon, Y. (2016). AndroZoo: Collecting millions of Android apps for the research community. In *Proceedings of the 13th International Conference on Mining Software Repositories* (pp. 468–471). ACM. <https://doi.org/10.1145/2901739.2903508>
- Feng, C., Huertas Celdrán, A., Sánchez Sánchez, P. M., Kreischer, J., von der Assen, J., Bovet, G., Martínez Pérez, G., & Stiller, B. (2025). CyberForce: A federated reinforcement learning framework for malware mitigation. *IEEE Transactions on Dependable and Secure Computing*, 22(4), 4398–4411. <https://doi.org/10.1109/TDSC.2025.3547005>
- Haque, M. A., Hossain, I., Kamol, M. M., Alam, M. J., Amalapuram, S. K., Talukder, S., & Rahman, M. S. (2026). LAMDA: A longitudinal Android malware benchmark for concept drift analysis. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=1FnCrZtBNQ>
- McMahan, H. B., Moore, E., Ramage, D., Hampson, S., & Agüera y Arcas, B. (2017). Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics* (Vol. 54, pp. 1273–1282). PMLR. <https://proceedings.mlr.press/v54/mcmahan17a.html>
- Ongun, T., Boboila, S., Oprea, A., Eliassi-Rad, T., Hiser, J., & Davidson, J. (2022). CELEST: Federated learning for globally coordinated threat detection [Preprint]. arXiv. <https://arxiv.org/abs/2205.11459>
- Sebastián, S., & Caballero, J. (2020). AVclass2: Massive malware tag extraction from AV labels. In *Proceedings of the 36th Annual Computer Security Applications Conference* (pp. 42–53). ACM. <https://doi.org/10.1145/3427228.3427261>
- Shen, J., Yang, W., Chu, Z., Fan, J., Niyato, D., & Lam, K.-Y. (2024). Effective intrusion detection in heterogeneous Internet-of-Things networks via ensemble knowledge distillation-based federated learning. In *ICC 2024 - IEEE International Conference on Communications* (pp. 2034–2039). IEEE. <https://doi.org/10.1109/ICC51166.2024.10622262>